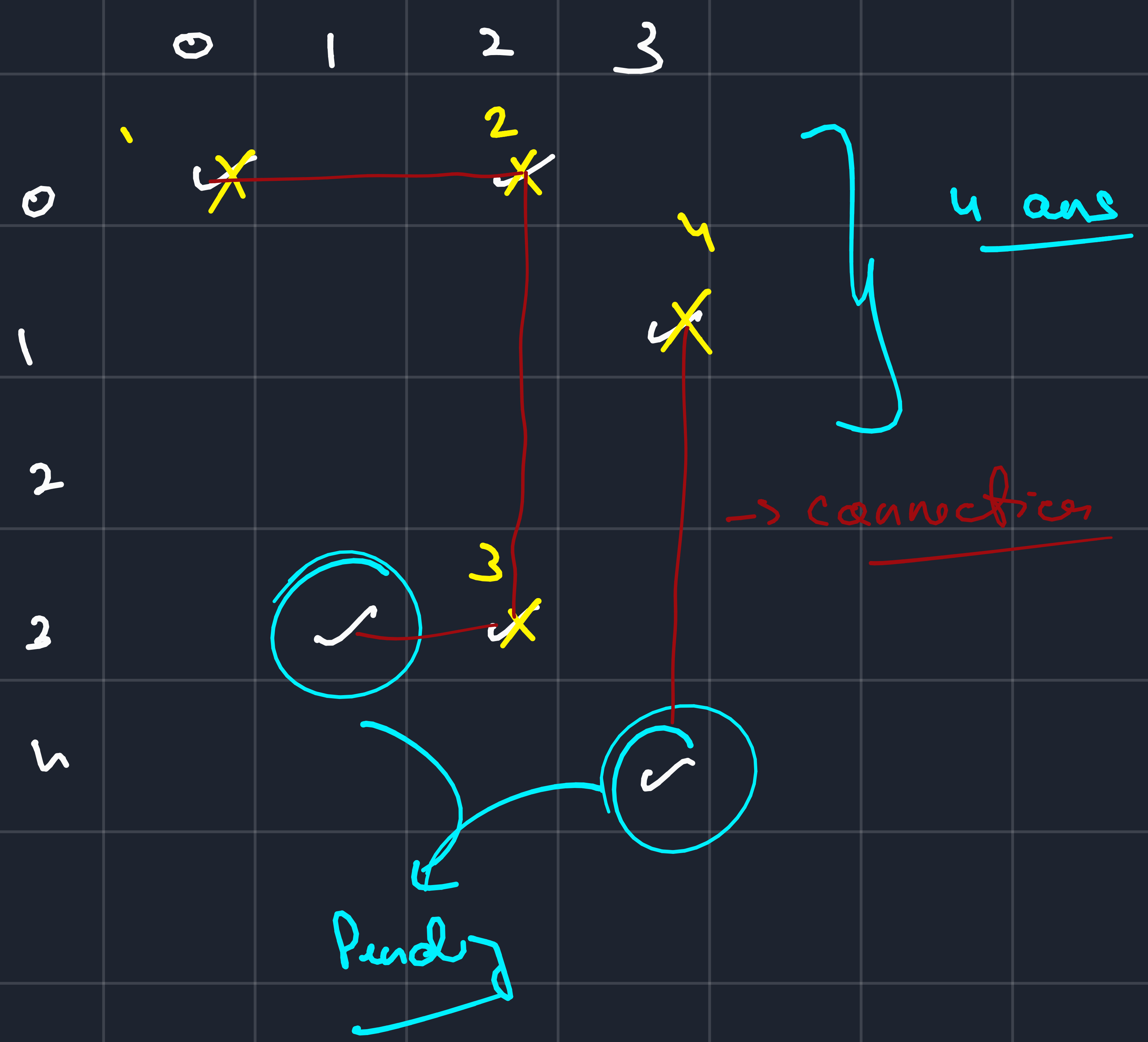
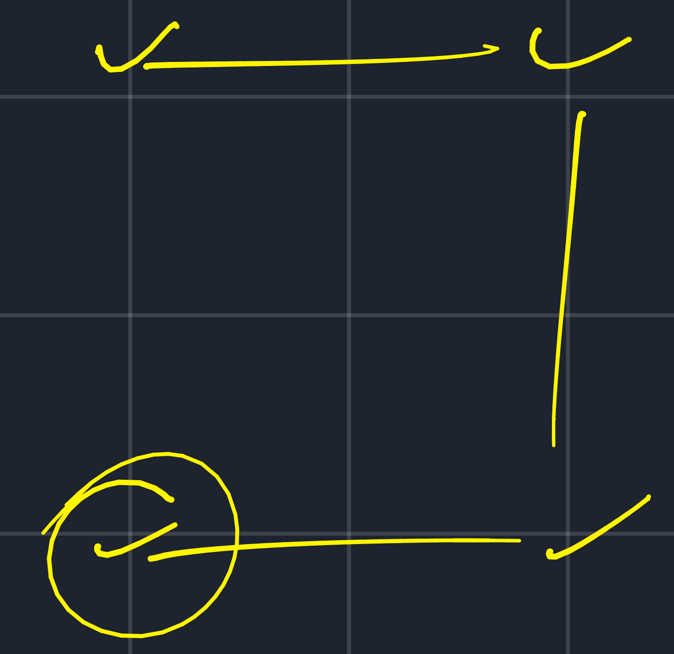
↳ maximum stones removed





↳ connect remove

↳ ans → (components size − 1)

check for all components

→ n - stones

components

c_1 c_2 c_3

x_1 → stones x_2 x_3

ans = $(x_1 - 1) + (x_2 - 1) + (x_3 - 1)$

$x_1 + x_2 + x_3 + \cdots = n$

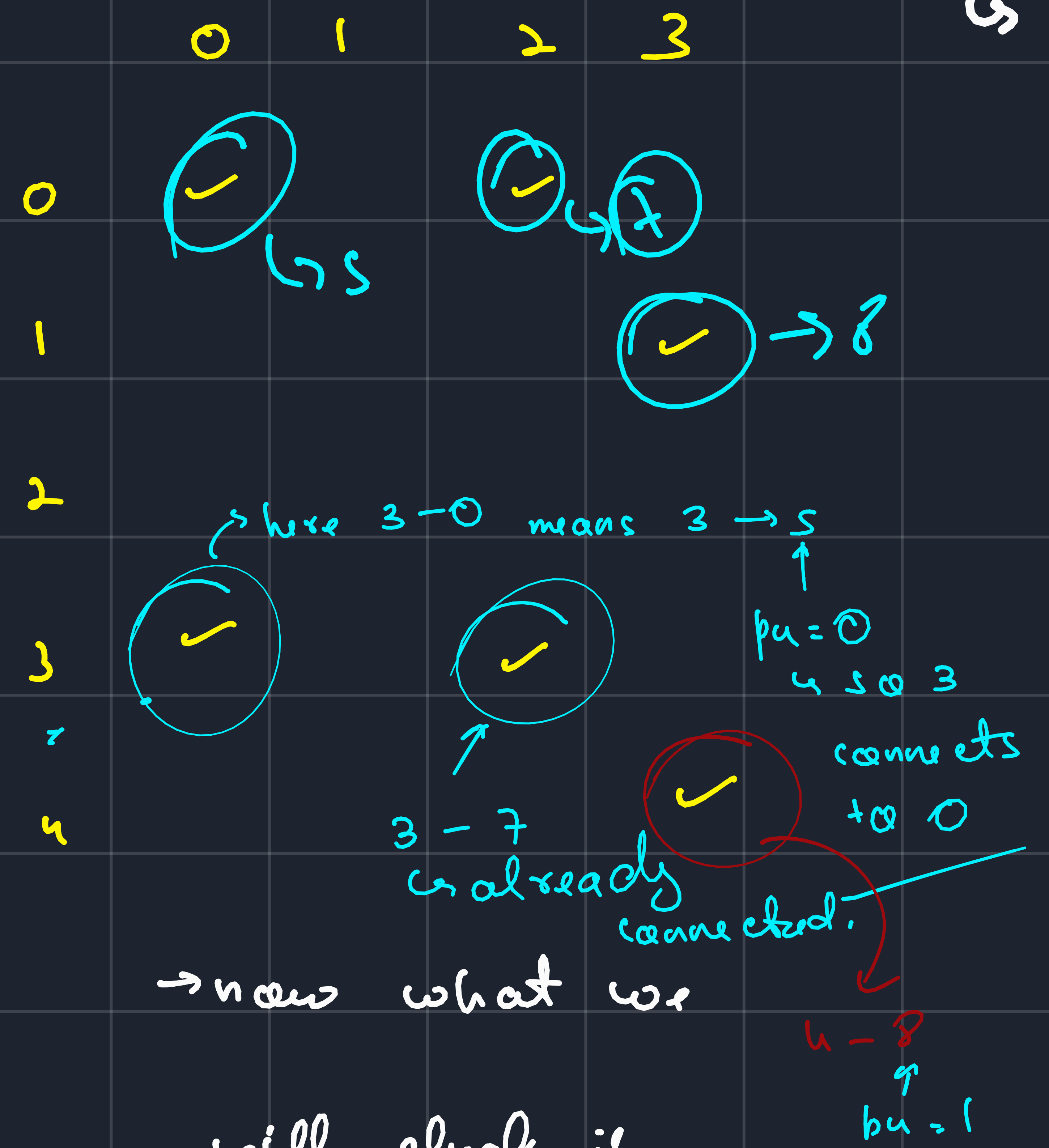
how many times ?

$(x_1 + x_2 + x_3) + \cdots - (1 + 1 + 1 + 1)$

ans = n - no. of components.

→ how to apply Disjoint sets.?

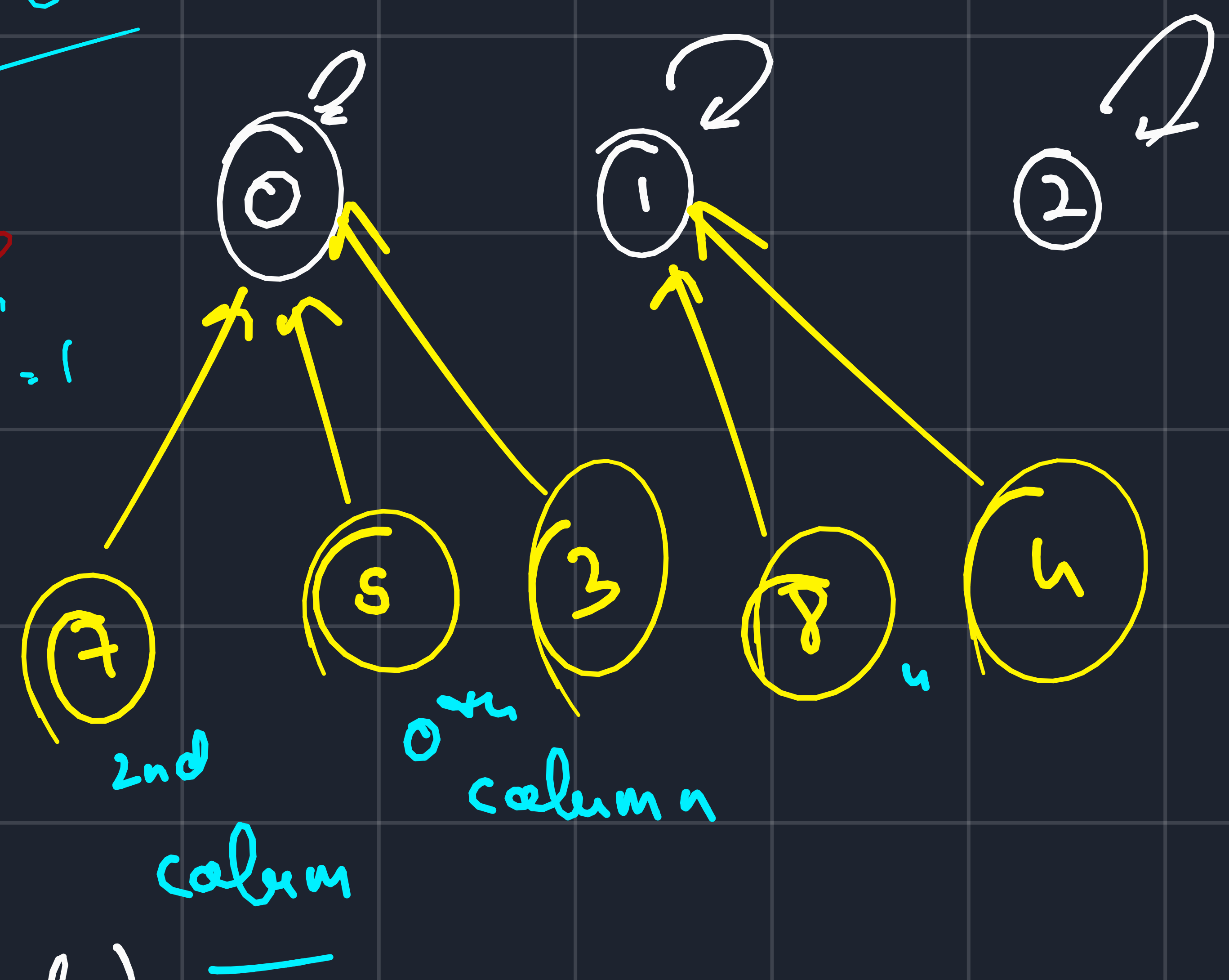
↳ Treat rows and col as node in Disjoint set.



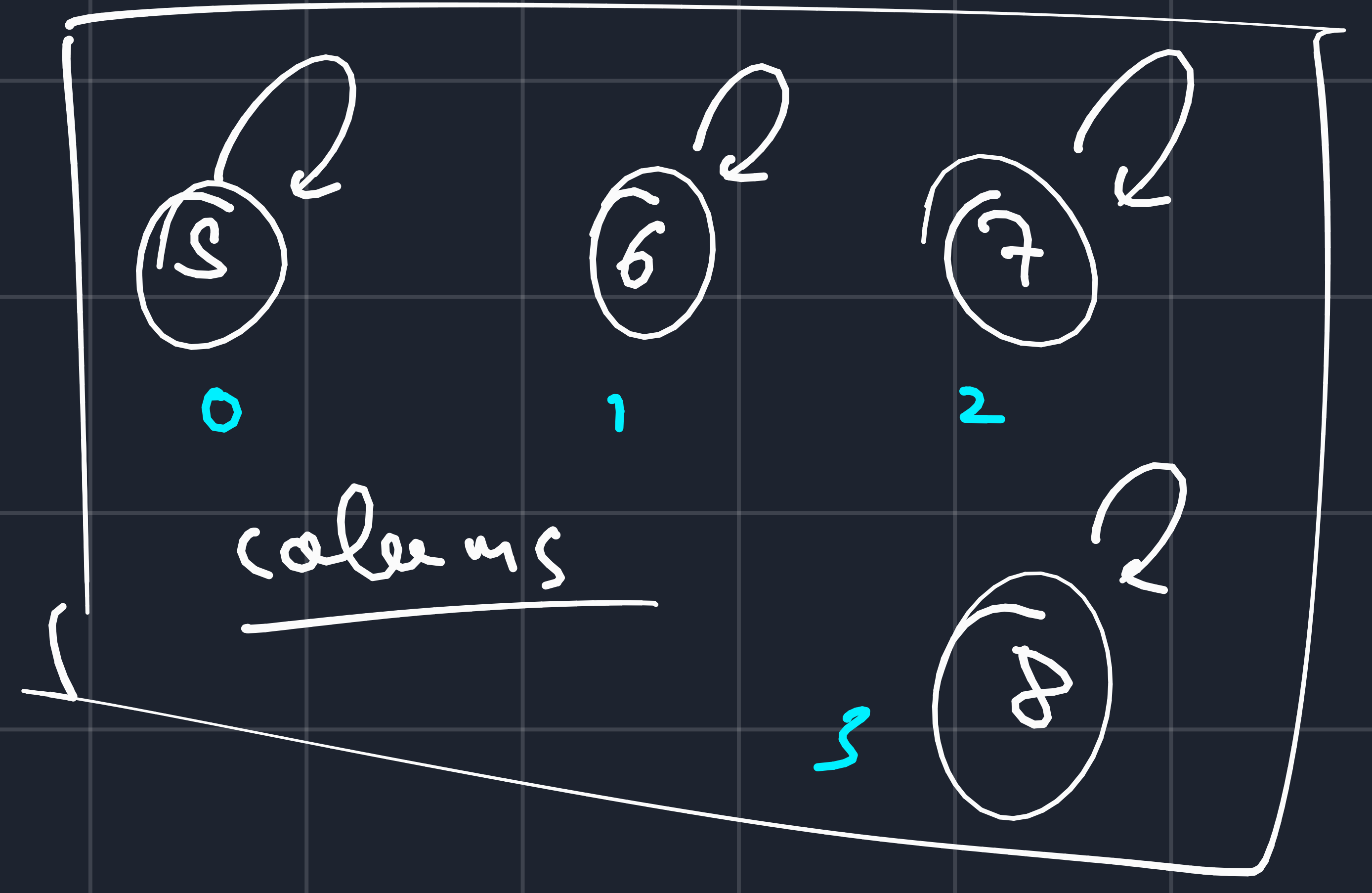
→ now what we will check is
↳ check the stone zero row
↳ combine with 0 colum. (5th node)

0 1 2 3 4 — rows

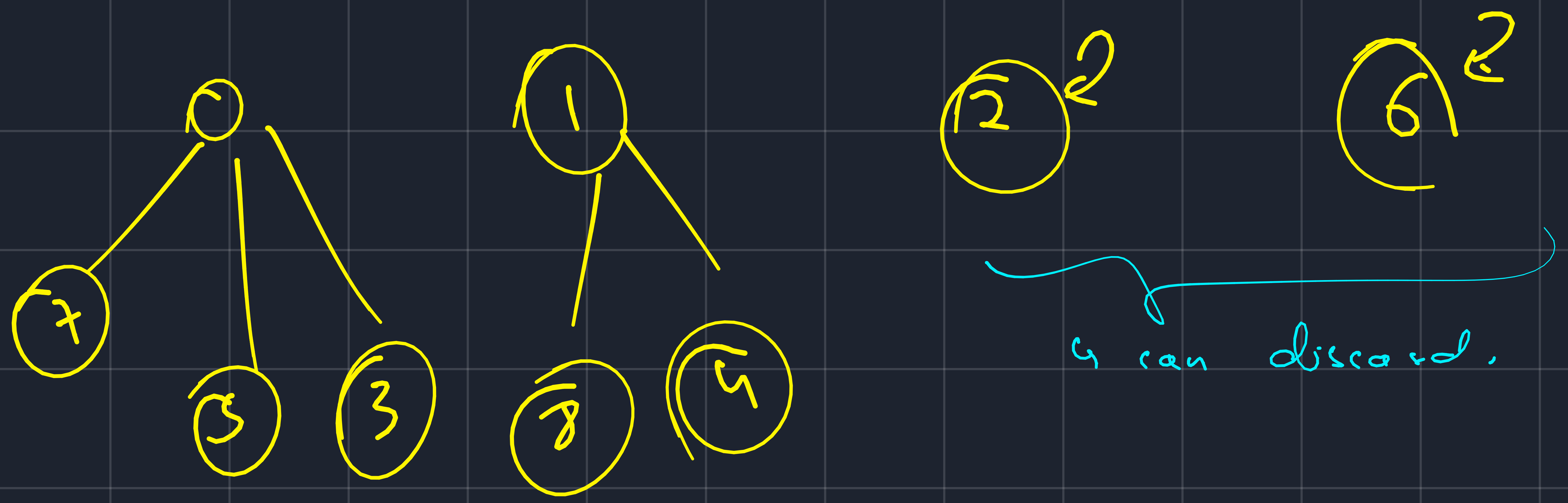
5 6 7 8 — col



→ columns mapped as nodes
↳ shift
↳ col + last val + 1
↳



Final components



↳ can discard.

↳ how to find valid components:-

↳ Take a stone and count it's ultimate parent.

↳ count the number of unique ultimate parents.